

Computing for Analytical Work

Session 1 — Where am I?

Block 2 — The terminal as a way to inspect reality

This block in one sentence

The terminal is not a different computer. It is a precise way to talk to the same computer.

- Same disk. Same folders. Same files you see in Finder or Explorer.
- The difference: you say exactly what you mean, and the computer tells you exactly what it did.
- Clicking hides both halves of that conversation.

Agenda

1. Two-minute recap: `pwd` , `ls` , `cd`
2. Shell, prompt, command, output — and the Windows rule
3. Making and moving things: `mkdir` , `cp` , `mv`
4. Looking inside files without Excel: `head` , `tail` , `wc`
5. Running a script with `uv run python ...`
6. Recording the commands that worked
7. Git thread: `git diff` . AI thread: ask before you run.
8. Stretch, Lab 2, Homework 1

Two-minute recap: `pwd`, `ls`, `cd`

```
pwd          # where am I?  
ls           # what is here?  
cd notebooks # go there; every relative path now starts from there  
cd ..        # up one level
```

You used all four in Lab 1. If any of them still feels foreign, tell a TA during the lab. We are not re-teaching them; we are building on them.

What a shell is

- The **terminal** is the window. The **shell** is the program inside it that reads what you type.
- The shell shows a **prompt**, waits, you type a **command**, press Enter, it runs it, prints the **output**, and shows the prompt again.
- That loop is the whole thing. Prompt, command, output, repeat.

```
anna@laptop project $ ls data/raw  
sales.csv  
anna@laptop project $
```

Anatomy of a command

```
ls -l data/raw
```

- `ls` — the **command**: the program to run.
- `-l` — an **option** (flag): changes how it behaves. Here, "long" listing with sizes and dates.
- `data/raw` — an **argument**: what to act on. A path, resolved from the cwd.

Spaces separate the pieces. A path with a space in it needs quotes: `"My Data/raw"`. Better: no spaces in project names.

Supported shells and the Windows rule

- **macOS / Linux:** the system Terminal (zsh or bash).
- **Windows: Git Bash**, installed with Git for Windows. PowerShell and `cmd.exe` are at your own risk; TAs will not debug them in lab.
- Every command in this course works identically in both.

Windows rule: never type bare `python` in Git Bash. It hangs silently. Always pass an argument:

```
python --version  
python -c "import sys; print(sys.executable)"
```

Commands that change things

`mkdir`, `cp`, `mv`

`mkdir` — make a folder

```
mkdir output          # one folder, in the cwd
mkdir -p data/processed # make parents as needed; no error if it exists
ls                    # confirm
```

- Relative to the cwd, like every other path.
- Programs do **not** create folders for you when they write files. A script that writes `output/summary.csv` into a missing `output/` fails. Remember this in the lab.

cp — copy

```
cp data/raw/sales.csv data/raw/sales_backup.csv    # file → new file  
cp -r data data_backup                             # folder → folder, recursively
```

- Source first, destination second. Both relative to the cwd.
- Copying makes **two** files. Now there are two truths. Before you copy data, ask whether you actually want that.

mv — move, which is also rename

```
mv notebooks/sales.csv data/raw/sales.csv    # move to the right place
mv analyze.py analyse.py                     # same folder: that is a rename
```

- One file remains. That is the honest fix for a misplaced file.
- **mv** will **silently overwrite** a file at the destination. Look first: `ls data/raw`.

Commands that only look

`head`, `tail`, `wc`

head — the first lines

```
head data/raw/sales.csv      # first 10 lines
head -n 3 data/raw/sales.csv # first 3
```

```
date,region,units,revenue
2026-01-03,North,12,1440.00
2026-01-03,South,7,840.00
```

You now know the delimiter, the header, the date format, and the column names. Before writing a line of code.

tail — the last lines

```
tail data/raw/sales.csv      # last 10 lines  
tail -n 2 data/raw/sales.csv # last 2
```

- Files often go wrong at the end: a truncated download, a trailing "Total" row, a blank line, a footer someone typed by hand.
- **head** and **tail** together tell you whether the file is *one* shape all the way through.

wc — how big is it?

```
wc -l data/raw/sales.csv      # count lines  
wc -l data/raw/*.csv         # all CSVs at once
```

```
1201 data/raw/sales.csv
```

1,201 lines, one of them a header: 1,200 rows. When your dataframe says 900, you have a number to argue with.

Why not just open it in Excel?

- Excel does not show you the file. It shows you its **interpretation** of the file.
- It reinterprets dates, drops leading zeros, converts `1,440.00` by locale, and on save may change the delimiter or encoding.
- Then you click Save, and the file on disk is different from the one you were given.
- `head` shows the bytes. It cannot change them.

Rule for the course: inspect with the terminal; analyse in Python; open in Excel only when you mean to edit.

Running a script

```
cd /path/to/project      # the root – where the README says to be
uv run python scripts/summarize.py
```

- `scripts/summarize.py` is a path, resolved from the cwd. So is every path *inside* the script.
- The script's cwd is **where you ran it from**, not where the `.py` file lives. Same rule as the notebook kernel in Block 1.
- Output goes wherever the script's relative paths point — from the cwd.

Sidebar: you have been typing `uv run` since setup

- The pre-course checklist had you run `uv sync` and `uv run python -c "import pandas; ..."`.
- Lab 1 had you run `uv run jupyter lab`. Lab 2 is `uv run python scripts/summarize.py`.
- For now it is a **recipe**: type it exactly, from the project root.
- Session 2, Block 3 opens by explaining what it has been doing all along. Until then, do not substitute bare `python`, and do not ask why. Just insist on it.

Recording the commands that worked

Keep a plain text file — `COMMANDS.md`, `notes.txt`, whatever — and paste in the exact lines that worked, in order, with the folder you were in.

```
# from the project root
uv sync
mkdir -p output
uv run python scripts/summarize.py
ls output
```

Not the ones you tried. The ones that **worked**. That file becomes your README's run instructions, and it is a homework deliverable.

Git thread — `git diff`: what did I change?

```
git status      # which files changed?  
git diff        # what changed inside them?
```

```
--- a/scripts/summarize.py  
+++ b/scripts/summarize.py  
-OUTPUT = "../output/summary.csv"  
+OUTPUT = "output/summary.csv"
```

Lines starting with `-` are what was there. Lines with `+` are what you wrote. Everything else is context.

Git thread — why `git diff` matters today

- Before checkoff, run `git diff`. If the diff shows more than your fix, you changed more than you meant to.
- The diff **is** your "what did you change?" answer. Paste it into the diagnosis note.
- Windows: if `git diff` shows every line of every file changed, `core.autocrlf` is not set to `input`. Fix it before you continue, or the diff is unreadable.

AI thread — ask before you run

I am in Git Bash on Windows, in the root of a Python project.
Before I run it, what does this command do, step by step?

```
rm -rf output/
```

Is there a version that only deletes CSV files, and asks before each one?

- **Explain before executing.** Especially anything with `rm`, `mv`, `-r`, `-f`, or `*`.
- **Ask for a safer version** of a destructive command. `rm -i`, a narrower pattern, a dry run.
- Then read the explanation, and decide.

AI thread — you are responsible for what runs on your machine

- The assistant did not press Enter. You did.
- "AI told me to" is not a diagnosis, not an excuse, and not something you can explain at checkoff.
- AI can help interpret commands and output. **You are responsible for what runs on your machine.**
- Good uses today: explain this command, explain this output, give me a safer version. Bad use: paste a block you cannot explain and run it.

5-minute stand-and-stretch

Stand up. Leave the laptop. Water, window, hallway.

Back in five minutes, then 45 minutes of lab. Homework 1 is on the last slide; it will still be there when you return.

Lab 2 — Run the project without clicking around

`scripts/summarize.py` reads a CSV with the standard library only — no pandas; this lab is about paths and the shell. The README says to run it from `scripts/`. It fails. Run it from the root, and it fails differently: `output/` does not exist. Fix both. Record the exact commands that worked. `git diff` to see your change. Diagnosis note. Checkoff.

From the project folder, type these exactly:

```
uv sync
uv run python scripts/summarize.py
```

Stretch: make the script create `output/` if missing. Why would this also run with bare `python scripts/summarize.py` and no `uv sync`? Hold that question for Session 2.

Homework 1 assigned — Files, paths, terminal, and run instructions

A repo that fails because files are misplaced, paths are wrong, or commands run from the wrong folder. Get it to run and document how. Expected time: **5–6 hours**. Due the **Friday after this session, 23:59** — slot on Moodle.

Deliverables, on Moodle:

1. The fixed repo, with corrected paths and the generated output file
2. A command log: the exact commands that worked, in order
3. A diagnosis note: symptom, cause, evidence, change, verification
4. Simple run instructions in the README
5. A **60–90 second video walkthrough**: screen recording, run the fixed code, explain what was broken, what you changed, how you verified. No script reading.